

Pointers and Arrays

Due at the start of your next lab period

Objectives

Use pointers to manipulate data in an array

Use const and define statements to improve flexibility and re-usability of functions

Requirements

Write a C program that creates an array of numbers and then prints and manipulates those numbers in various ways.

Your program must:

1) Create the array of **N_NUMS** consecutive values starting at **START** having the data type **WORD**, where these three boldfaced entities must be appropriately declared using **#define**, **typedef**, or **const** (you should use each of these). The array is declared and initialized in the main().

2) Contain the following functions:

```
/* Swap the two values pointed at by x and y (no loop here) */  
void swap (WORD *x, WORD *y);
```

```
/* Print all of the values in a 1-D array where the array size is N  
 * and WORD_FORMAT has been #define'd */  
void printVector (const WORD *vec, const int N);
```

```
/* Print all of the values in a 1-D array one byte at a time where the  
 * array size is N.  
 * Prints an extra space after each word */  
void printBytes (const WORD *vec, const int N);
```

optional function

```
/* Print all of the values in a 1-D array one byte at a time in reverse  
 * byte order for each word where the array size is N.  
 * Prints an extra space after each word */  
void printBytesReverseEndian (const WORD *vec, const int N);
```

3) printVector() and printBytes() must use pointer notation with post-increment

4) printBytesReverseEndian() must use pointer notation

5) After creating the array, the main() will then:

- Print the contents of the array using printVector()
- Swap adjacent numbers by calling swap() with pointer notation (no square brackets)
- Print the contents of the array using printVector()
- Swap the numbers back by calling swap() with array notation (yes, square brackets)
- Print the contents of the array using printVector()
- Print the bytes within the data using printBytes() with extra space between WORDs.
- Optional: Print the bytes in reverse-endian order with extra space between WORDs.

6) All functions (including main()) can be in one file

7) You must demonstrate that your **#defines** and **consts** work properly by recompiling the program to handle a different data type and number sequence.

Hints

For printing, an unsigned character is a half of a half of an unsigned integer

```
printf("%hhu", myUchar);
```

The data type and the conversion string for printing must be maintained as a pair:

```
typedef int WORD;           // Must be compatible with next line
#define WORD_FORMAT "%d " // Must be compatible with previous line
```

```
#define N_NUMS 10
```

An example of `const`:

```
const WORD START = 1011;
```

If you put these in a separate header file, make them `static const`.

You will need to use the compile-time function `sizeof()` for for your byte-printing functions.

Evaluation

A working, well-documented program	8/10
Correct use of #define	1/10
<code>printBytesReverseEndian()</code>	1/10

Sample Run

```
dale$ ./pointerLab
Original Vector:
1011 1012 1013 1014 1015 1016 1017 1018 1019 1020
Adjacent words swapped:
1012 1011 1014 1013 1016 1015 1018 1017 1020 1019
Words swapped back:
1011 1012 1013 1014 1015 1016 1017 1018 1019 1020
Data Bytes:
243 3 0 0 244 3 0 0 245 3 0 0 246 3 0 0 247 3 0 0 248 3 0 0 249 3 0
0 250 3 0 0 251 3 0 0 252 3 0 0
Word bytes with endian reversal:
0 0 3 243 0 0 3 244 0 0 3 245 0 0 3 246 0 0 3 247 0 0 3 248 0 0 3
249 0 0 3 250 0 0 3 251 0 0 3 252
```